

PRD: Lead Extractor (v0)

Documento de produto pra ferramenta interna de extração de leads B2B via Google Maps + scraping.

Versão: 0.1 **Autor:** Natan (Grupo Dupla / Klicka) **Status:** rascunho pra implementação **Deploy alvo:** GitHub + Vercel

1. Objetivo

Ferramenta interna que combina Google Places API + scraping HTML pra transformar uma busca tipo "Dentistas Niterói" em lista de leads B2B enriquecida com email, WhatsApp e Instagram, exportável pra CSV pronta pra cadência de outbound.

Sem fila, sem DB, sem auth no v0. Deploy Vercel + repo GitHub. Uso interno.

2. Usuário

Natan + irmão. Uso interno. Se virar pública depois, adiciona auth.

3. Fluxo de uso (uma tela, 3 estados)

Estado 1: Busca

- Barra livre no topo: "Dentistas Niterói", "Clínicas de estética Copacabana", etc
- (Opcional v1) Filtros de tipo + cidade lado a lado como atalho
- Botão "Buscar"

Estado 2: Lista bruta (resultado do Places)

- Tabela com N linhas (até 60 por query)
- Colunas: checkbox, Nome, Categoria, Endereço, Rating, Telefone, Site
- Ações: select all / select none, contador "X selecionados"
- Botão "Extrair contatos dos selecionados"

Estado 3: Lista enriquecida

- Nova tabela com os selecionados + colunas adicionais: Email, WhatsApp, Instagram, Status do scrape

- Botão "Exportar CSV"
- Botão "Voltar" pra refinar seleção

4. Funcionalidades V0

- Busca via Google Places API (New) Text Search
- Enrichment via Places API Place Details (Contact tier: phone + website)
- Scraping HTML dos websites via fetch + cheerio
- Extração via regex/DOM query: email, telefone, WhatsApp (wa.me/api.whatsapp.com), Instagram
- Detecção de tipo de link: se site é wa.me direto, extrai número sem scraping
- Batch de scrape em chunks de 10 sites por request pra caber no timeout Vercel Hobby (10s)
- Export CSV com papaparse

5. Fora do escopo V0 (explicitamente)

- Auth / multi-usuário
- Dedupe automático (visual only na tabela)
- Filtros complexos ou salvamento de busca
- Persistência em DB (Vercel Postgres fica pra v1)
- Scraping de Instagram/Facebook (só guarda o handle)
- Sites JS-heavy (SPAs) via headless browser
- Enrichment via Hunter/Snov
- Rate limit / throttling pro usuário
- Histórico de leads extraídos
- Marcar lead como "contatado"

6. Arquitetura

Uma única página Next.js, dois API routes serverless:

```
Cliente (React)
├─ Busca      —POST /api/search—► Places API
├─ Seleção    (só estado local)
└─ Enrich     —POST /api/enrich—► fetch + cheerio (paralelo)
```

Estado da sessão fica no client (Zustand ou useState). Zero persistência no v0.

7. Stack

- **Framework:** Next.js 14+ (App Router, TypeScript)

- **UI:** Tailwind + shadcn/ui (Button, Input, Table, Checkbox, Toast)
- **State:** useState + useReducer, ou Zustand se ficar complexo
- **HTTP:** fetch nativo
- **HTML parse:** cheerio
- **CSV:** papaparse
- **Deploy:** Vercel (Hobby serve, upgrade pra Pro se timeout apertar)
- **Repo:** GitHub, deploy automático no push pra main

8. Especificação dos endpoints

POST /api/search

Input:

```
{ "query": "Dentistas Niterói" }
```

Comportamento:

1. Chama Places API Text Search com `textQuery: query`, retorna até 20 places por página
2. Pagina até 3 páginas via `nextPageToken` (delay 2s entre páginas)
3. Pra cada place_id, chama Place Details com fields `displayName,formattedAddress,rating,userRatingCount,nationalPhoneNumber,websiteUri,types`
4. Normaliza e devolve

Output:

```
{
  "results": [
    {
      "placeId": "ChIJ...",
      "name": "Clínica Odontológica X",
      "category": "dentist",
      "address": "Rua Y, 123, Niterói",
      "rating": 4.7,
      "reviewCount": 128,
      "phone": "+55 21 99999-0000",
      "website": "https://clinicax.com.br"
    }
  ],
  "totalFound": 42
}
```

POST /api/enrich

Input:

```
{
  "items": [
    { "placeId": "...", "name": "...", "website": "https://..." }
  ]
}
```

Comportamento:

1. Roda em paralelo com `Promise.allSettled`
2. Timeout de 5s por site (`AbortController`)
3. Detecta tipo de URL:
 - `wa.me/*` ou `api.whatsapp.com/send*` : extrai phone do URL, retorna direto
 - `instagram.com/*` : extrai handle do path, retorna sem raspar
 - `facebook.com/*` : idem
 - `linktr.ee` , `bio.link` , `beacons.ai` : raspa e devolve todos os links classificados
 - Outro (site próprio): raspa HTML com cheerio
4. Extractors HTML:
 - Email: regex `[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}` + blocklist (wordpress.com, elementor.com, sentry.io, wixpress.com, godaddy.com)
 - WhatsApp: procura hrefs com `wa.me/` , `api.whatsapp.com/send?phone=` , ou `whatsapp://send?phone=`
 - Instagram: procura hrefs com `instagram.com/` (ignora `/p/` , `/reel/`)
 - Telefone extra: hrefs com `tel:`

Output:

```
{
  "results": [
    {
      "placeId": "...",
      "email": "contato@clinicax.com.br",
      "whatsapp": "+5521999990000",
      "instagram": "@clinica_x",
      "scrapeStatus": "ok" | "no_website" | "timeout" | "parse_failed" | "empty"
    }
  ]
}
```

Limite: máximo 15 itens por request (segurança pro timeout). O client chunca lista maior antes de mandar.

9. UI (spec das telas)

Layout único, tela cheia, max-width 1200px, cabeçalho fixo.

Cabeçalho: logo/título "Lead Extractor" à esquerda, badge de crédito Google Places gasto no mês à direita (v1).

Estado 1 (busca)

- Input grande centralizado, placeholder "Ex: Dentistas Niterói"
- Botão "Buscar" primário
- Abaixo do input, dica "Tipo de estabelecimento + cidade ou bairro. Até 60 resultados por busca."

Estado 2 (lista bruta)

- Barra superior: "X resultados encontrados. Y selecionados." + botão "Nova busca" (secundário) + botão "Extrair contatos" (primário, desabilitado se Y=0)
- Tabela: checkbox header (select all), Nome, Categoria, Endereço curto, Rating (X reviews), Telefone, Site (link clicável)

Estado 3 (enriquecido)

- Barra superior: "X leads extraídos." + botão "Exportar CSV" (primário) + botão "Voltar" (secundário)
- Tabela: Nome, Telefone, Email, WhatsApp, Instagram, Status (badge colorido: verde ok, amarelo empty, cinza no_website, vermelho timeout/parse_failed)

Loading states: skeleton na tabela durante `/api/search`. Barra de progresso "Extraindo 12/40..." durante enrich (feito em chunks no client).

10. Variáveis de ambiente

```
GOOGLE_MAPS_API_KEY=xxx # server-only, nunca prefixar NEXT_PUBLIC
```

Config no Google Cloud Console:

- Habilitar: Places API (New)
- Restringir key por HTTP referrer (o domínio Vercel + localhost) OU por IP (se rodar só server)
- Setar orçamento máximo de \$50/mês com alerta em \$25 pra evitar surpresa

11. Roadmap de execução (dia 1)

Sequência realista pra bater fim de dia:

Bloco	Tempo	O que fica pronto
1	30min	Repo GitHub, <code>create-next-app</code> , shadcn/ui setup, deploy vazio no Vercel funcionando

Bloco	Tempo	O que fica pronto
2	60min	<code>/api/search</code> + Places API integração + teste com curl
3	60min	UI estado 1 e 2 (busca + tabela de resultados)
4	90min	<code>/api/enrich</code> com cheerio + regex + chunking no client
5	60min	UI estado 3 + CSV export + loading states
6	30min	Polish (empty states, error toasts, dica de uso)

Total: ~6h focadas. Cabe num dia se travar a agenda.

12. Backlog v1 (post-MVP, priorizado)

1. Vercel Postgres pra persistir buscas e leads
2. Dedupe automático (fuzzy match em nome + cidade + telefone normalizado)
3. Filtros: bairro, categoria, rating mínimo
4. Marcar lead como "contatado" / "convertido" na tabela
5. Auth (Clerk ou next-auth) pra abrir pro irmão
6. Enrichment via Hunter/Snov (email do dono via domínio)
7. Detecção de sites JS-heavy + fallback pra @sparticuz/chromium
8. Histórico de buscas com filtro por tag/vertical
9. Integração direta com Instantly/Smartlead via API
10. Job assíncrono pra scrape grande (fila via Upstash QStash)

13. Riscos e mitigações

Risco	Mitigação
Vercel Hobby timeout 10s	Chunk de 10 a 15 sites por request <code>/api/enrich</code> ; client orquestra múltiplas requests
Rate limit / IP block em scraping	User-Agent realista, timeout curto (5s), sem retry agressivo, se virar problema migrar pra ScrapingBee
Key Google exposta	Só server-side, restringir por referrer, orçamento máximo no Cloud
Falso positivo de email	Blocklist de domínios de framework/host; heurística "email tem que estar em página do domínio raspado, não externa"
Site JS-heavy retorna vazio	Marca <code>scrapeStatus: empty</code> , usuário sabe que precisa abrir manual, v1 fallback com headless
Custo Google Places subir	Cache no Vercel KV por 30 dias de queries repetidas (v1)